

RAG技术的三大范式和技术演进

从基础到前沿，彻底掌握增强检索生成



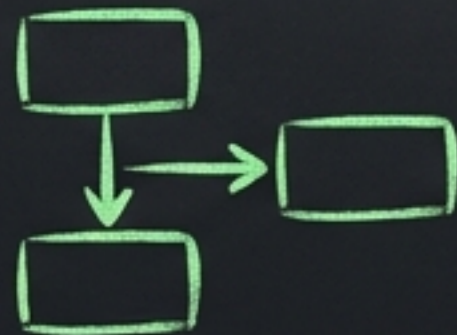
我们的探索之旅：RAG知识地图



第一站：大模型的“记忆”难题
(为什么需要RAG?)

1

2



第二站：Naive RAG诞生
(RAG是如何工作的?)



第三站：精益求精的演进之路
(RAG如何变得更强大?)

4



第四站：总结与工具箱
(我们学到了什么?)

才华横溢，但有时会“一本正经地胡说八道”

为什么我们不能只用LLM？

知识截止



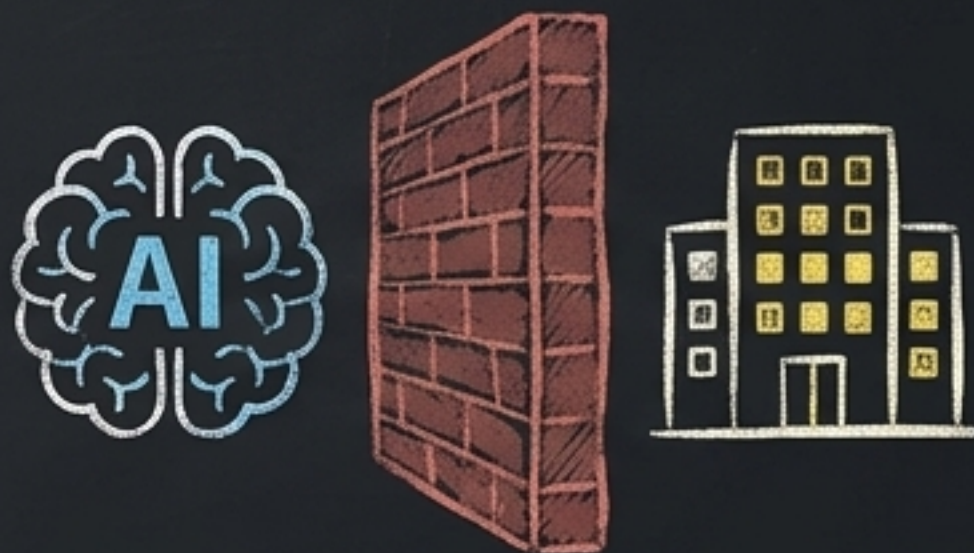
模型所知有限，
无法获取最新信息。

模型幻觉



在缺乏知识时，会自信地编造
答案。这是最关键的问题。

私域盲点



无法访问企业内部、
非公开的专业知识。

结论：独立的LLM在真实性、实时性和私密性方面无法满足企业级应用的需求。

RAG：给大模型一场“开卷考试”

核心思想：RAG (Retrieval-Augmented Generation)，中文是“增强检索生成”。先检索，再生成。



关键信息：不再依赖模型的“记忆”，而是让它根据我们提供的“参考资料”来回答问题。

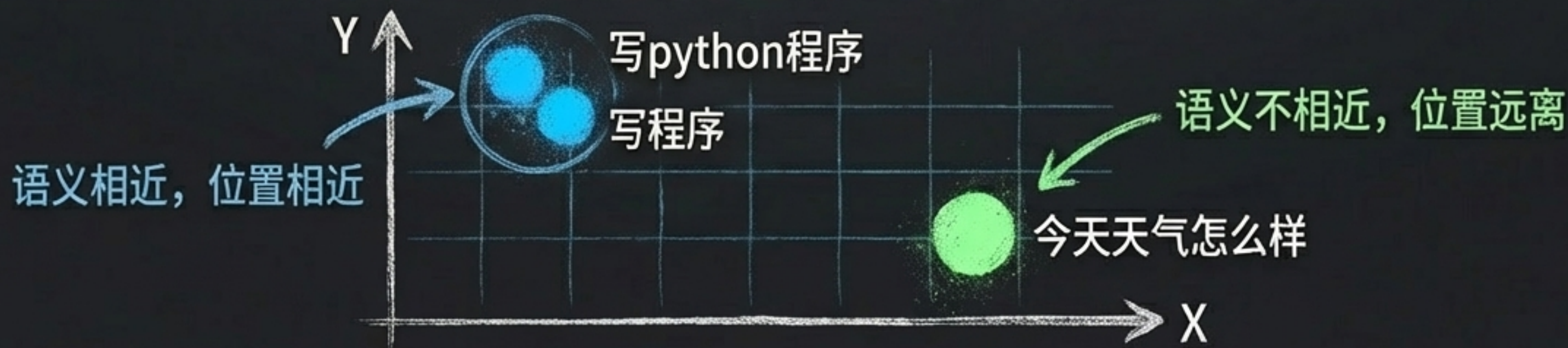
RAG的核心引擎：让机器理解语义的“密码”

什么是Embedding?

Embedding模型就像一个“语义编码器”。输入任意文本，输出一个固定长度的数字数组（向量）。



核心特性：语义相近的文本，其向量在多维空间中的位置也相近。



第一步（离线）：构建你的专属知识库

流程：知识的预处理（The Indexing Pipeline）



1. 加载与分块（Load & Chunk）

将长文档加载进来，并切分成更小的、独立的知识片段（Chunks）。

2. 向量化（Embed）

将每一个知识片段都通过 Embedding 模型转换成向量。

3. 存储（Store）

将文本片段和它们对应的向量一起存入向量数据库。

关键组件



Pinecone



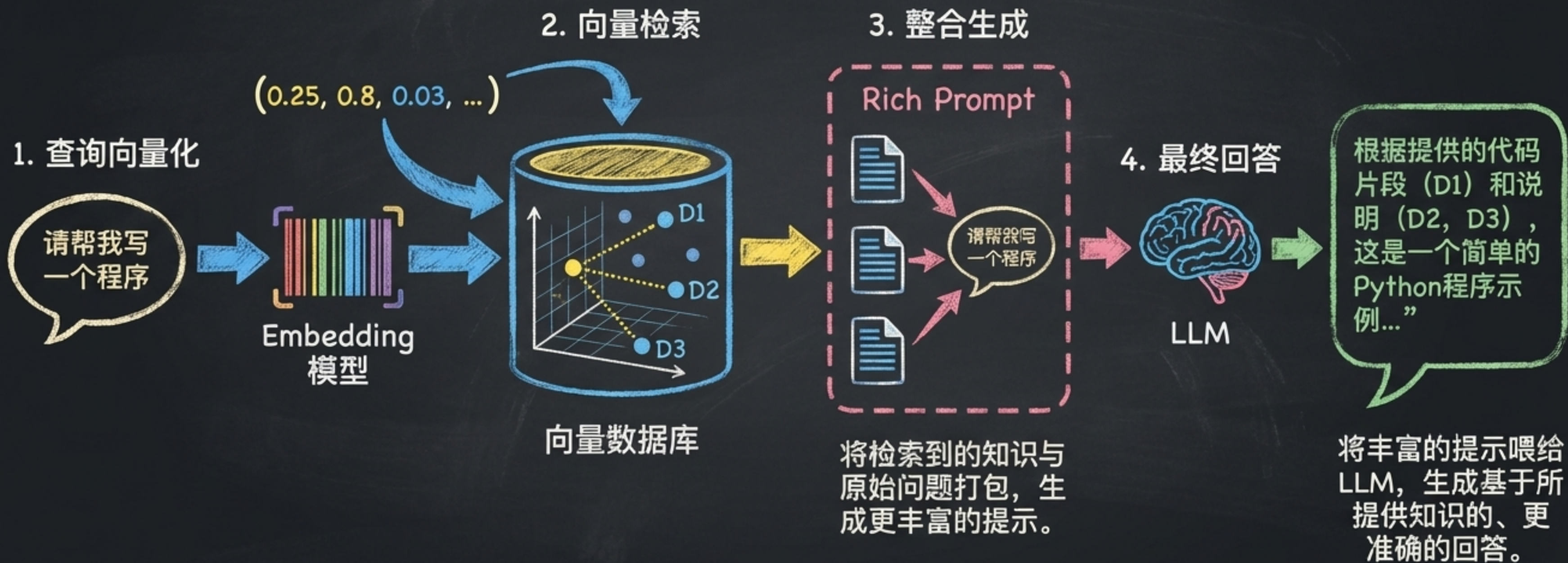
Chroma



pgvector

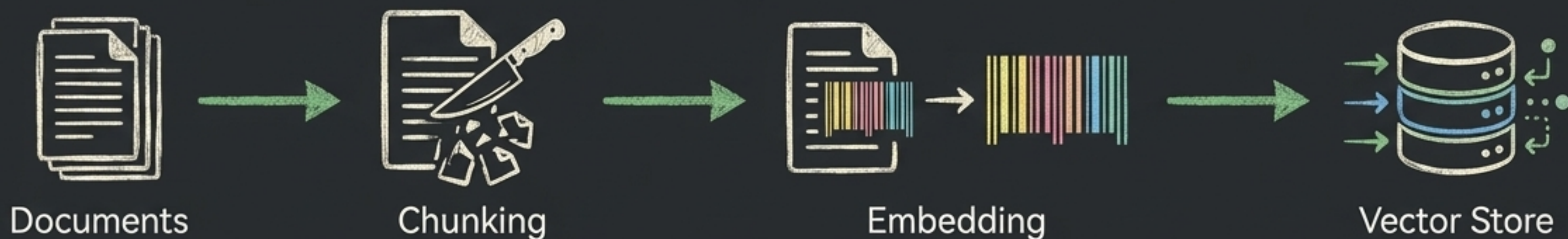
第二步（在线）：从提问到精准回答

流程：实时检索与生成 (The Retrieval & Generation Pipeline)

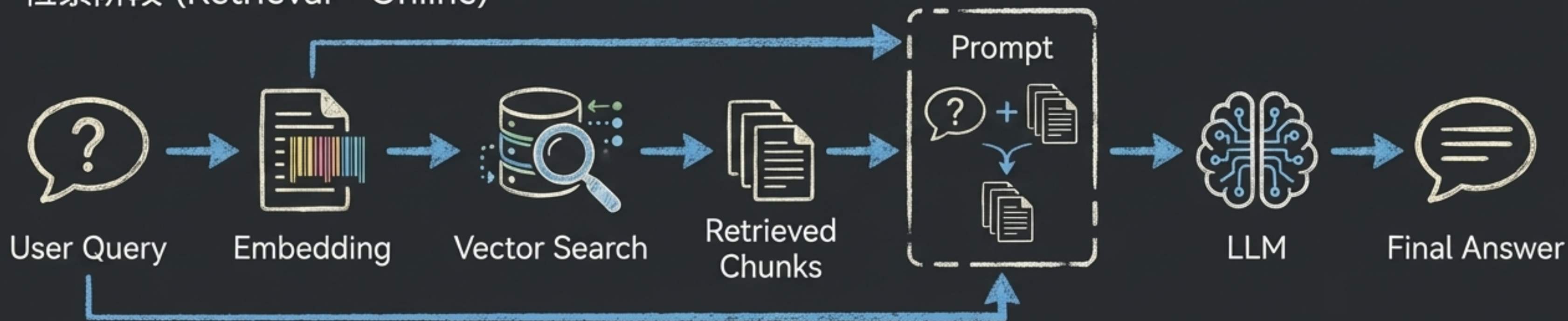


范式一：经典RAG (Naive RAG) 架构全景

索引阶段 (Indexing - Offline)



检索阶段 (Retrieval - Online)

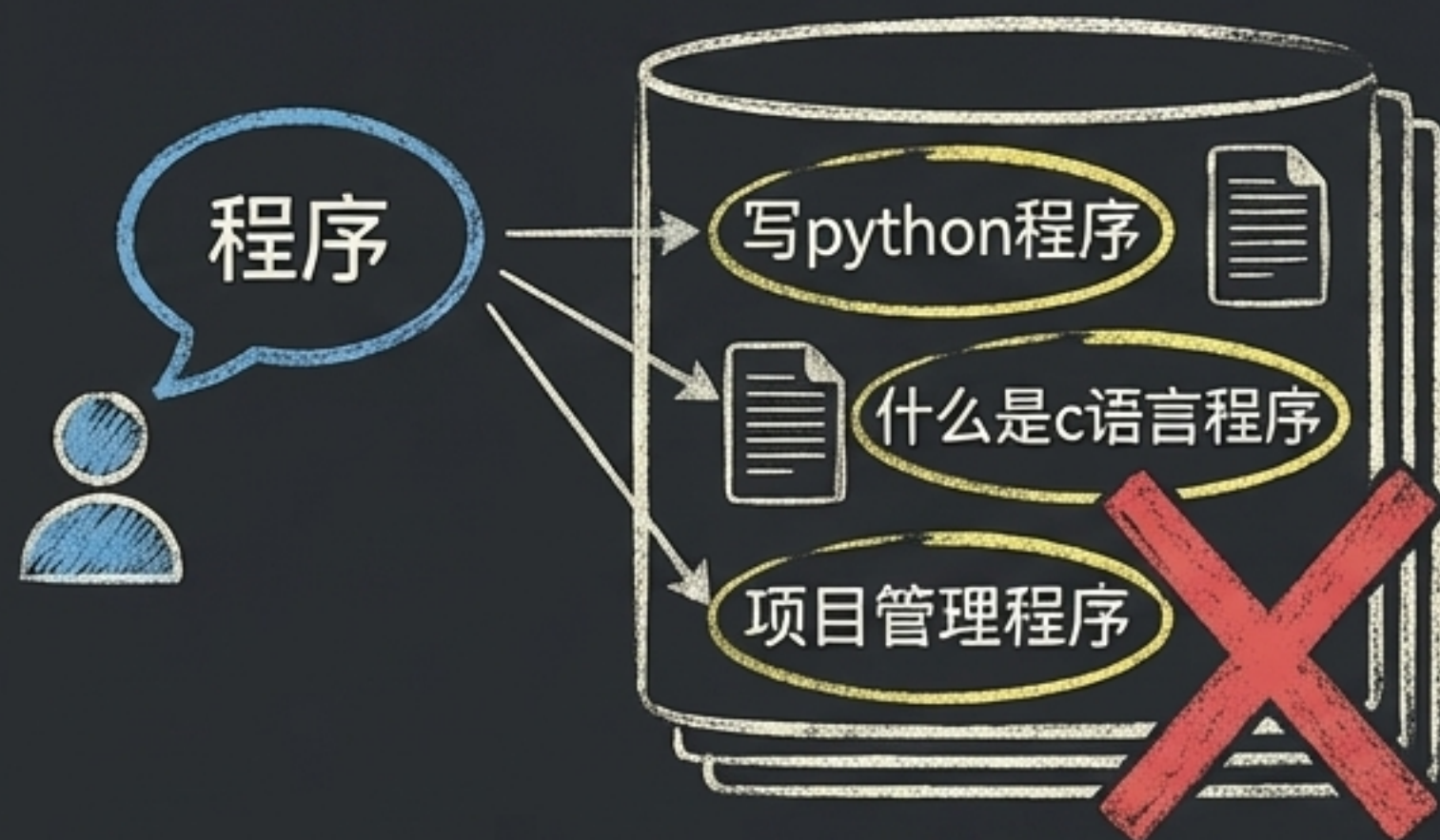
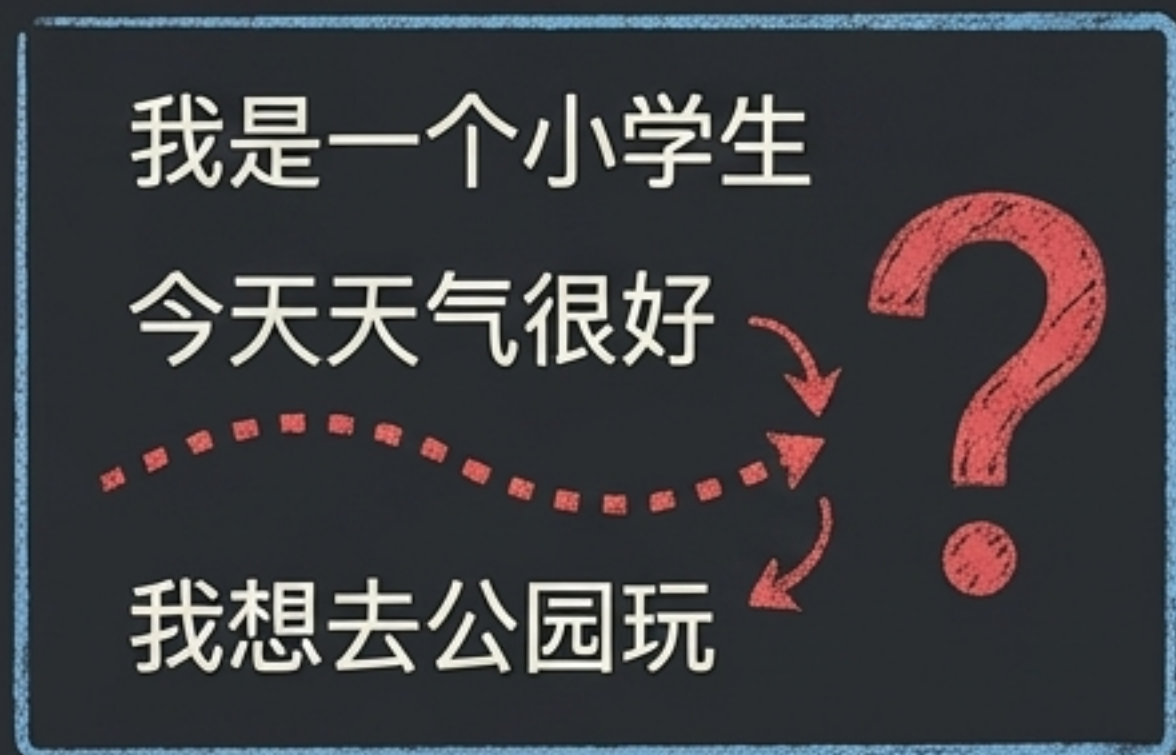


Naive RAG 的“成长的烦恼”

“RAG其实并不是一个非常完美的东西”

分块的困境 (Chunking Problem)

检索的难题 (Retrieval Problem)



固定的分块大小会破坏语义完整性。太大的块包含噪音，太小的块丢失上下文。



“失之毫厘，谬以千里”。检索结果的质量直接决定了最终答案的质量。

范式二：高级RAG（Advanced RAG） – 追求极致的“精准”

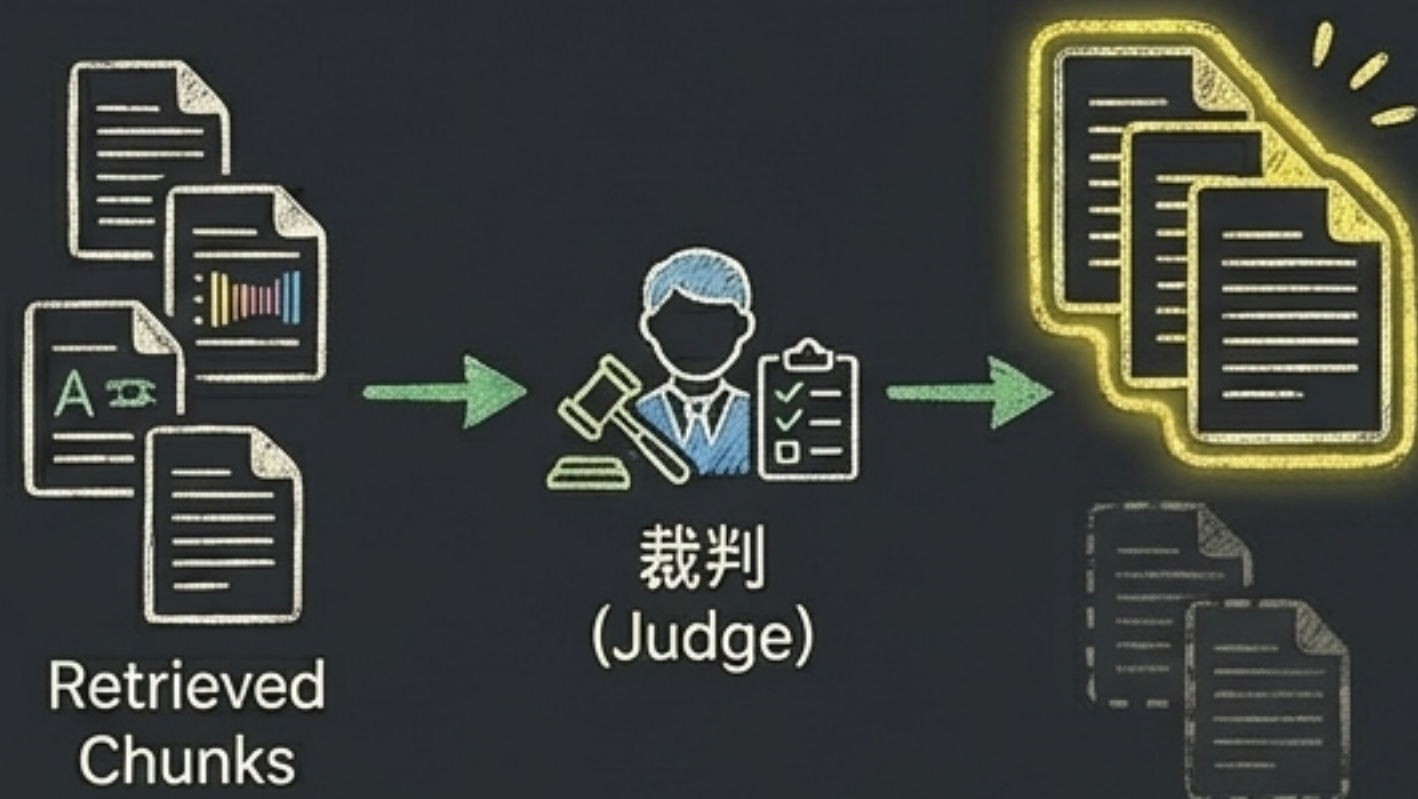
核心思想：在检索前后增加“质检”和“优化”环节，确保喂给LLM的都是高质量的“食材”。

检索前优化（Pre-retrieval）



将用户的模糊问题，改写或扩展成更具体、更适合机器检索的查询。

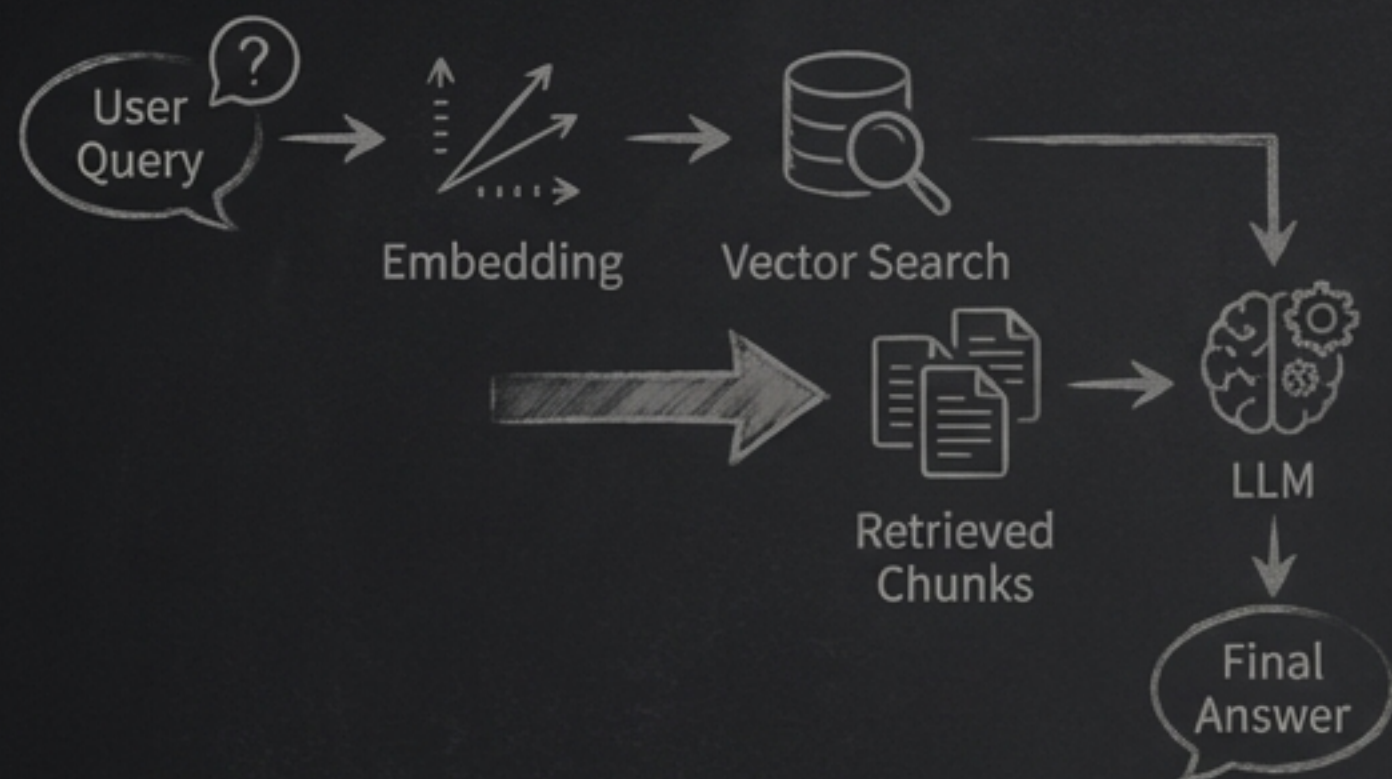
检索后优化（Post-retrieval）



用一个更轻量、更精准的模型，对初步检索出的文档进行重新排序，把最相关的放在最前面。

架构升级：从“能用”到“好用”

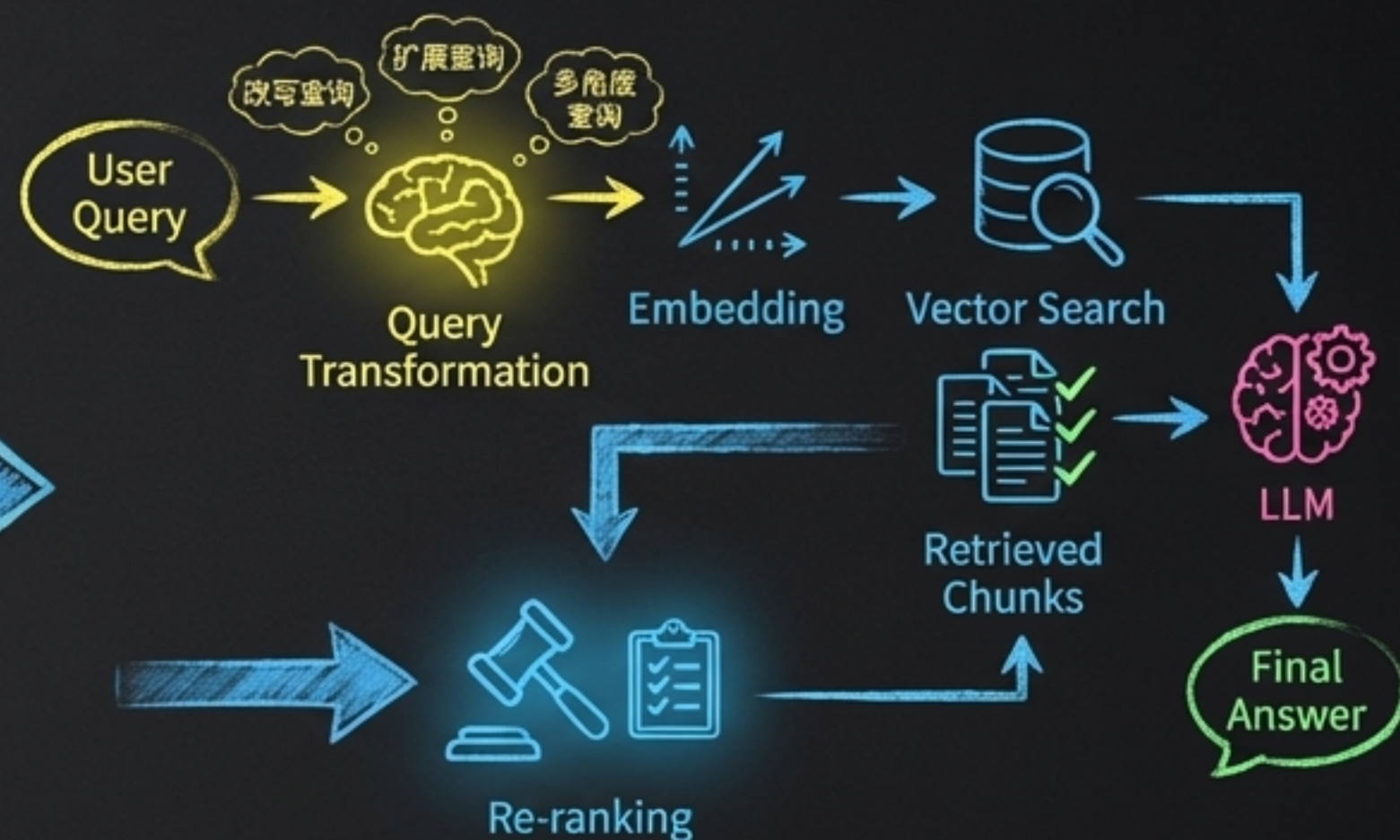
Naive RAG



演进



Advanced RAG



Naive RAG 是一个基础架构，直接将用户查询转化为向量，在向量数据库中搜索最相关的文本块，然后将这些块和查询一起发送给 LLM 生成答案。

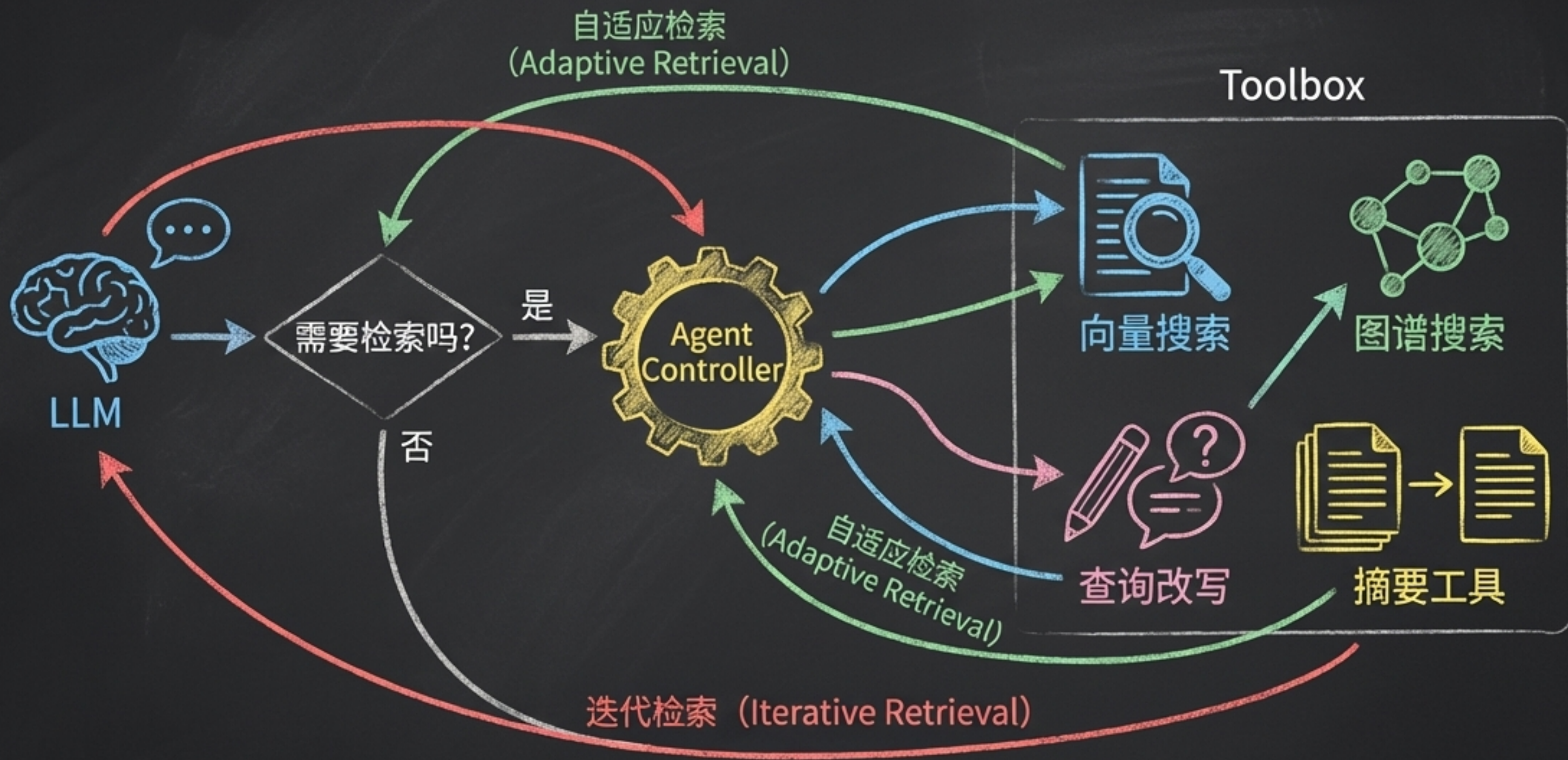
问题：可能因为分块和检索的不准确，导致最终答案的质量不稳定。

Advanced RAG 引入了两个关键优化模块：Query Transformation（查询转换）和 Re-ranking（重排序）。

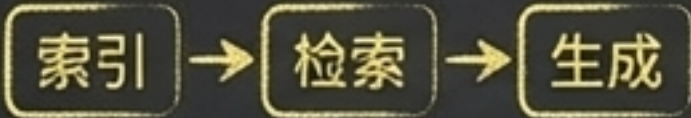
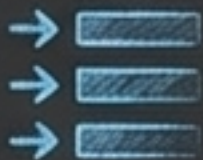
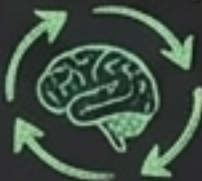
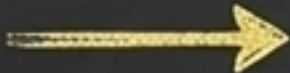
改进：通过优化查询和精选检索结果，显著提升了 LLM 获取信息的质量，从而生成更精准、更有用的答案。

范式三：模块化RAG（Modular RAG） - 迈向“自动化与智能化”

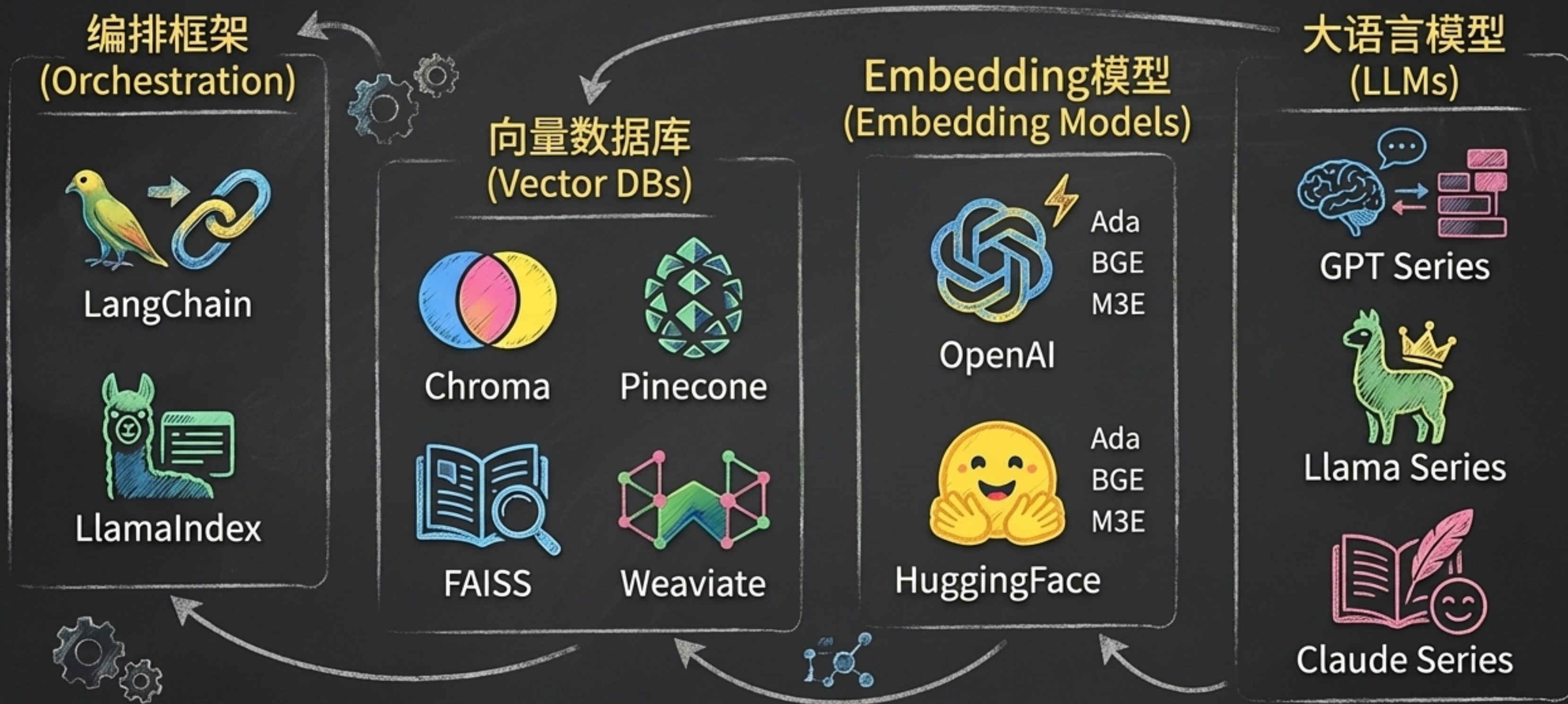
核心思想：打破固定的线性流水线，将RAG的各个组件视为可自由组合、按需调用的“工具”或“模块”。



一图胜千言：RAG三大范式演进路径

维度 (Dimension)	Naive RAG (基础框架)	Advanced RAG (精准优化)	Modular RAG (智能灵活)
核心思想	索引-检索-生成 	优化检索质量 	可插拔、自适应流程 
关键技术	基本向量检索 	重排序、查询重写 	Agent、迭代/图检索 
流程	线性、固定 	优化的线性流程 	动态、非线性、自适应 
适用场景	PoC验证、简单问答 	生产级知识库、客服 	复杂研究、自动化任务 

程序员的工具箱：构建你的第一个RAG应用



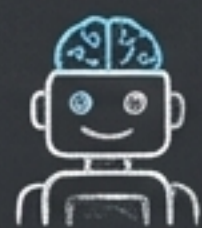
核心启示：为AI打造一个更强大的外接“大脑”



RAG是解决LLM幻觉和知识局限性的最有效“外挂”。它将LLM从一个“封闭记忆者”变成一个“开卷研究员”。



RAG的演进，本质上是追求更精准的上下文和更智能的流程。从“能找到”到“找得准”，再到“知道该怎么找”。



未来是模块化与Agent的深度融合。RAG系统将不再是简单的问答工具，而是能够自主规划、执行复杂任务的智能助理。



给开发者的建议：从 Naive RAG 快速上手，但从第一天起，就用模块化、可扩展的思想去设计你的系统。